

dataQ 전체 기획 및 개발 방향

작성 기준 시점: 2026-04-11 작성 방식: 2026-03-11 최초 커밋부터 현재까지의 개발 이력을 되짚어, 무엇을 왜 만들었는지 / 어디로 가고 있는지를 기획 관점에서 정리

이 문서는 특정 기능의 설계서가 아니다. dataQ라는 제품이 왜 이렇게 생겼고, 앞으로 어디로 가는지를 개발자 본인의 작업 이력 위에서 설명하는 기획 성격의 글이다. 그동안 설계 문서가 파편적으로 쌓인 터라, 한 번 전체를 훑는 관점이 필요했다.

1. dataQ가 풀려는 문제

1-1. 실무에서의 출발점

조직이 커지면 데이터베이스가 여러 개가 되고, 테이블이 수천 개가 된다. 여기서 두 가지 문제가 반복된다.

- **같은 의미의 컬럼이 시스템마다 다르게 생긴다.** 어떤 시스템은 `USR_ID`, 다른 시스템은 `USER_ID`, 또 다른 곳은 `MEMBER_ID`. 한글 COMMENT도 제각각이다. 이게 쌓이면 데이터 통합·이관·분석이 전부 어려워진다.
- **표준이 있어도 지켜지지 않는다.** 표준사전을 만들어 놓아도, 개발자 입장에서는 거기에 맞춰 컬럼을 만드는 것이 부담이고 ("매번 담당자에게 물어봐야 한다"), 담당자 입장에서는 이미 만들어진 수천 개 컬럼을 점검할 방법이 없다. 표준이 문서로만 존재하는 상태.

dataQ는 이 두 문제를 "자동화로" 풀겠다는 도구다. 담당자가 표준사전을 관리하면, dataQ가 외부 DB들을 주기적으로 스캔해서 표준 준수 여부를 진단하고, 문제가 있는 곳을 지목하고, 해결까지 이어지게 만드는 것.

1-2. 이 방향이 왜 유효한가

경쟁사로는 SMETA, WISE META, MetaStream 같은 메타데이터 관리 도구들이 있다. 이들의 핵심은 메타데이터 수집과 카탈로그화다. dataQ는 그것을 깔고 가되, 한 발 더 나아가 "표준 vs 현실의 gap"을 정량화하고 해소하는 데 초점을 맞췄다. 진단 결과의 준수율을 대시보드로 보여주고, 이슈를 해결 경로로 연결하고, 개발자가 새 테이블을 만들 때 표준에 맞게 쓰도록 자동 추천한다. 이 두 방향이 현재까지의 차별점이다.

2. 제품 구성의 뼈대

아키텍처 레벨에서는 세 모듈로 쪼개져 있다.

- **q-common** — VO, MyBatis 매퍼, 공용 유틸(DB 접속, 엑셀 파싱, 암호화). 나머지 두 모듈의 공용 라이브러리. 빌드 시 jar로 말려서 나머지 모듈의 lib/에 들어간다.
- **q-center** — 웹 포탈(8080). REST 컨트롤러, 화면(Vue 2 + Vuetify), 인증(Spring Security). 사용자가 보는 모든 것.
- **q-executor** — 백그라운드 실행기(8090). 외부 DB 스캔, 표준화 진단, 구조 진단, 대량 일괄 작업. 실행 중 진행률은 WebSocket(STOMP/RabbitMQ)으로 q-center에 전송해서 화면에 띄운다.

이 구조를 처음부터 그렇게 잡은 이유는 단순하다. 웹 요청 스레드에서 무거운 배치가 돌면 안 된다. 외부 DB를 스캔하는 데 몇 분이 걸릴 수 있고, 진단 하나에 수만 개 컬럼이 지나가면 수십 초가 걸린다. q-executor를 별도 프로세스로 떼어내서 웹 응답과 배치 실행을 분리했다.

기능 영역은 아래의 여섯 축으로 나뉜다. 각 축이 어떻게 발전해 왔는지가 이 기획 문서의 본문이다.

1. 외부 DB 수집 (데이터 모델)
2. 표준 사전 관리
3. 표준화 진단
4. 자동 표준화 지원
5. 구조 진단
6. 승인·변경이력·협업

3. 각 기능 축이 걸어온 길

3-1. 외부 DB 수집 (데이터 모델)

2026-03-11의 첫 커밋부터 있던 축이다. 초기 관심사는

"Oracle/PostgreSQL/MySQL/MSSQL/MariaDB/Cubrid 같은 여러 DBMS를 일관된 방식으로 읽어온다"는 것이었다. 이를 위해 TB_DATA_SOURCE에 접속 정보(호스트/포트/사용자/암호)를 Jasypt로 암호화해서 저장하고, 수집 시 각 DBMS에 맞는 SQL을 갖고 JDBC로 접속해 테이블·컬럼·길이·타입·COMMENT를 긁어온다.

수집 결과는 TB_DATA_MODEL_CLCT(수집 이력) → TB_DATA_MODEL_OBJ(테이블) → TB_DATA_MODEL_ATTR(컬럼) 계층으로 저장된다. 같은 모델을 여러 번 수집하면 이력이 쌓이고, "최신 수집 기준" 외에 "과거 어느 시점 기준"으로도 볼 수 있게 된 것이 나중에 구조 진단으로 이어진다.

개발 이력에서 이 축은 굵직하게 다음을 거쳤다.

- 수집 화면 설계와 기본 흐름 (2026-03-23)
- 멀티 DBMS 검증: 9종 JDBC 드라이버 확인 (2026-04-01)
- SID/ServiceName 구분, Oracle 접속 변형 지원 (2026-04-08)
- 스키마 필터(TB_DATA_MODEL_SCHEMA) 조회 개선 (2026-04-08)
- 인덱스·제약조건까지 수집할 것이냐는 숙제가 설계서에 남아 있음 (아직 미구현)

3-2. 표준 사전 관리

dataQ의 가장 오래된 축이자, 가장 많은 리팩토링이 들어간 영역이다. 구조는 이렇게 생겼다.

```
TB_WORD      - 단어 (사용자, 이름, 번호, ...). 영문명/영문약어/도메인분류
TB_TERMS     - 용어 (사용자이름). 단어들의 조합 + 도메인 연결
TB_TERMS_WORDS - 용어와 단어의 매핑 (순서 포함)
TB_DOMAIN    - 도메인 (일자도메인, 금액도메인, ...). 데이터타입/길이/단위
TB_CODE_INFO + TB_CODE_DATA - 코드 체계 (추후 확장)
TB_APPROVE_STAT - 승인 상태 (용어·도메인·단어·코드 전부에 다형적으로 붙음)
TB_WORD_DICT - 추천 사전 (TB_WORD에는 없지만 "자주 보이는 일반 단어"를 미리 깔아둔 사전)
```

이 구조의 핵심은 **용어가 단어와 도메인에 의존**하고, **단어가 도메인분류에 의존**한다는 것이다. 그래서 "용어 하나 등록"이 순진하게는 "단어 등록 → 도메인 등록 → 용어 등록"의 연쇄가 된다. 이 체인 의존성이 UX 문제를 만들었고, 자동 표준화 축(3-4)이 탄생한 직접 원인이 되었다.

이력에서 이 축의 흐름:

- 기본 CRUD와 일괄 엑셀 업로드 (초기)
- 용어 등록 시 단어 검증 강화, 일괄등록 검증 추가 (2026-03-27)
- DSTerm.vue(용어 화면) 분석 후 추가/수정 모달 코드 중복 정리 속제 도출
- 영문약어 규격 강화: [A-z0-9] 만 허용, 언더바 금지 (2026-04-02)
- 금칙어·유사어 검증 로직 (2026-04-01)
- 동의어/이음동의어: PostgreSQL 배열(ALLOPH_SYNM_LST)을 UNNEST로 풀어 검색
- TB_WORD_DICT 대량 시드: 337건 → 3,475건 → 48,159건 (2026-04-02 하루에 집중적으로 확장)

3-3. 표준화 진단

표준사전을 기준으로 외부 DB의 컬럼을 자동 검사한다. 진단 유형은 다섯 가지.

- TERM_NOT_EXIST: 컬럼 한글명·영문명 둘 다 표준 용어에 없음
- TERM_KR_NM_MISMATCH: 영문명만 있고 한글명(COMMENT)이 표준과 다름
- TERM_ENG_NM_MISMATCH: 한글명만 있고 영문 약어가 표준과 다름
- DATA_TYPE_MISMATCH: 컬럼 타입이 도메인 표준과 다름
- DATA_LEN_MISMATCH: 컬럼 길이가 도메인 표준 범위를 벗어남

진단은 q-executor에서 비동기로 돈다. 시작 시 TB_DIAG_JOB을 INSERT하고 (status=READY), q-executor가 runDiag를 받으면 RUNNING으로 바꾸고, 각 컬럼을 돌면서 TB_DIAG_RESULT에 건건이 저장한다. 완료 시 DONE, 중간에 중지 요청이 오면 stopFlags ConcurrentHashMap을 통해 STOPPED로 마무리한다. q-center UI는 3초 폴링으로 processCnt/status를 확인한다.

이 축의 특이점은 **"READY → RUNNING" 사이에 사용자가 중복 클릭을 하는 버그가 반복적으로 나왔다**는 것이다. 실행 로직 분석 문서를 별도로 만들어서, READY 상태의 의미("데이터는 로드됐지만 처리 시작 전")와 중복 클릭 방지, 폴링 간격 최적화, KOMORAN 토큰 성능 이슈 등을 정리했다.

대시보드(3-7)에서 준수율을 뽑을 때의 소스도 여기였다. 초기엔 TB_DATA_MODEL_STATS 기반이었다가 TB_DIAG_JOB 기반으로 바뀌었다 (2026-04-01).

3-4. 자동 표준화 지원

이 연구일지(01 ~ 04)가 자세히 다룬 영역이다. 기획 관점에서 요약하면.

왜 만들었나. 진단은 이미 만들어진 테이블의 이슈를 찾아낸다. 하지만 이슈가 수백 건 나와도 해결 경로가 화면 이동 + 수동 입력이라서 실제로 고쳐지지 않는다. 용어 등록 자체도 단어·도메인 의존 체인 때문에 부담이 크다. 게다가 개발자 입장에서는 테이블을 만드는 시점에 표준을 알고 싶지, 만든 다음에 고치고 싶지 않다.

어떤 흐름으로 풀었나. 네 단계의 위저드.

1. 입력 — 한글 컬럼명을 엑셀/텍스트로 업로드
2. 자동 분석 — 파이프라인(기존 용어 완전매칭 → OKT 형태소 → 단어 매칭 → 영문약어 조합 → 도메인 추천 → 상태 판정)
3. 리뷰 — 분석 결과 테이블. AUTO는 승인만, PARTIAL은 미등록 단어 정보만 채우고 승인, FAILED는 수정 modal
4. 일괄 등록 + DDL 생성 — 단어 선등록 → 용어 등록 → 선택된 DBMS용 DDL 스크립트 출력

기술적으로 가장 많이 손댄 구간. 2단계의 단어 분리 알고리즘. 일지 04에서 자세히 썼듯이, greedy로 시작했다가 테슬라코일제목 반례를 만나 DP로 전면 교체했다. 현재는 dpSplit이 주 경로, greedySplit은 fallback으로만 남아 있다.

이 축이 차별성을 만들었다. 경쟁사 분석 결과, "한글 컬럼명 엑셀 업로드 → 단어 분리 → 영문 표준명 조합 → DDL 생성"까지 이어지는 자동 추천 흐름은 SMETA/WISE/MetaStream에는 없었다. 이게 dataQ의 고유 기능으로 경쟁사 분석 보고서에 명시적으로 실렸다.

3-5. 구조 진단

표준화 진단이 "컬럼 단위 표준 준수"를 본다면, 구조 진단은 "시간에 따른 스키마 변화"를 본다. 수집 이력 두 개(예: 지난달 수집 vs 오늘 수집)를 비교해서 테이블/컬럼의 추가·삭제·변경을 찾아낸다. 데이터 마이그레이션, 스키마 drift 모니터링 용도.

이 축은 2026-03-31 하루에 설계→구현→수정→리팩토링이 압축적으로 일어났다. 커밋 로그가 말해준다.

- 구조 진단(GAP 분석) 신규 개발
- 수집 스냅샷 vs 라이브 DBMS 직접 비교
- 수집 이력 기반 비교로 근본 수정
- 구조 진단 2차 개선: 수집 이력 선택 UI + 스키마 비교 뷰어
- 구조 진단 메뉴 통합 (structDiag 별도 메뉴 → 구조 진단으로 통합)
- 전면 리팩토링: q-center에서 q-executor로 이동, 대메뉴 분리
- compareSchema 버그 수정, TB_DATA_MODEL에 결과 저장
- 수집 폴링 버그 수정(기존 완료건을 새 수집으로 오인)
- prev_collect_dt 타입 캐스팅(VARCHAR → TIMESTAMP) 수정
- StructDiagService import 수정

하루에 열 번 이상 고친 이유는 구조 진단이 수집·진단·UI 세 영역에 걸쳐 있어 경계가 흐릿했기 때문이다. 처음엔 q-center에서 돌렸다가 무거워서 q-executor로 넘겼고, 메뉴 체계도 "스키마 비교" → "구조 진단"으로 rename했다. 이후 2026-04-07에 "구조변경진단 재설계"라는 한 번 더 큰 리팩토링이 들어간다. 이 축은 아직 완성됐다고 보기 어렵다.

3-6. 승인·변경이력·협업

표준사전이 한 사람만 고치는 게 아니라 여러 사람이 협업으로 고치는 환경을 전제한다. 그래서 승인·이력·협업 축이 필요하다.

- **승인 프로세스:** TB_APPROVE_STAT 한 테이블에 OBJ_TYPE + OBJ_ID의 다형 참조로 용어/도메인/단어/코드 승인을 모두 담는다. 역할 기반(작성자/검수자/승인자), 상태 전이(DRAFT → REVIEW → APPROVED/REJECTED), 거부 의견 저장, 감사 기록. 2026-04-01 단일 날짜에 설계→구현→테스트→비관리자 테스트까지 한 번에 돌렸다. 이후 2026-04-02 ~ 04-03에 "내 요청 현황" 메뉴, 반려/재요청 흐름, 영문명 컬럼 추가 등이 붙었다.
- **변경 이력:** TB_CHANGE_HISTORY로 누가 언제 무엇을 이전값에서 새값으로 바꿨는지 기록. 2026-04-01 설계→구현 연속.
- **커뮤니티 게시판:** 사용자 간 Q&A와 공지. 2026-04-02 재설계 포함. 참여 저조가 초기 진단되어 개선 작업이 이어짐.
- **마이페이지:** 2026-04-07. 내 활동, 내 승인 요청, 내 진단 이력.

이 축은 "단일 기능"이라기보다는 여러 축을 가로지르는 UX 레이어다. 데이터 표준이 운영에 들어가면 반드시 협업 도구가 필요하다는 문제 인식.

3-7. 대시보드 (부수 축)

위 여섯 축에 더해 대시보드를 따로 본다. 운영자 관점에서 dataQ에 들어왔을 때 가장 먼저 보는 화면이기 때문에, 숫자를 어떻게 보여주느냐가 "dataQ가 쓸 만한 도구인가"의 첫인상이 된다.

이력:

- 첫 구현에서 도넛 3개로 준수율을 표시했으나 의미가 겹쳐서 "표준 준수율 + 구조 일치율" 2개로 축소 (2026-04-02)
- 도넛 크기 200 → 180 → 재조정
- 진단 준수율 추이 라인차트 추가
- 데이터 모델 선택 드롭다운 추가, '전체' 옵션 제거 후 첫 모델 자동 선택
- 준수율 소스 변경: TB_DATA_MODEL_STATS → TB_DIAG_JOB 기반
- GROUP BY 에러 수정

대시보드는 "설계 완료"라기보다 "데이터가 쌓이면서 계속 조정되는 화면"으로 두고 있다.

4. Phase 구분 (이력상의 계획)

이력을 시간 순으로 한 번 더 큰 덩어리로 묶으면 이렇다.

Phase 0 — 기반 구축 (2026-03-11 ~ 2026-03-24)

초기 수집, 진단 결과 화면, 기본 CRUD. 이 시점까지는 "프로토타입"에 가깝다. 디자인·편의 기능이 거의 없다.

Phase 1 — 핵심 기능 완성 (2026-03-27 ~ 2026-04-01)

표준화 추천 시스템 신규 개발, 표준화 추천 고도화, 편의성 개선사항 34건 적용, 테스트 커버리지 137 → 205 → 229건, 구조 진단 신규 개발, 승인 프로세스 강화 설계·구현·테스트, 변경 이력 추적, 사용자 매뉴얼 작성. 경쟁사 분석 보고서에서 "Phase 1 100% 달성"이라 명시한 지점이 여기. 핵심 기능이 이 구간에 집중적으로 들어갔다.

Phase 2 — 확장 (2026-04-01 ~ 2026-04-02)

영향도 분석, ERwin 모델 임포트, 논리/물리 데이터 모델 구분, 전역 통합 검색, 멀티 DBMS 검증. 경쟁사 분석 보고서에서 "Phase 2 80%" → "Phase 2 완료"로 단계적으로 갱신. 메타데이터 관리 도구로서의 기능 폭을 넓힌 구간.

Phase 3 — 알고리즘과 품질 (2026-04-02 ~ 2026-04-03)

DP 기반 단어 분리, 블랙박스 테스트 104건(최종 97.6% 통과), 잠재적 버그 분석, 표준화 추천 검증 강화, 승인 반려/재요청 흐름. 기능 추가보다는 "이미 있는 기능의 견고함을 올리는" 구간.

Phase 4 — 재설계 및 정리 (2026-04-07 ~ 2026-04-09)

구조변경진단 재설계, 마이페이지, 메뉴 정리, 데이터소스 SID/ServiceName 처리, 게시판 개선, 스키마 조회 개선. "Phase 1에서 서둘러 만들었던 부분을 제대로 다시 짜는" 구간. 구조 진단이 가장 큰 리팩토링 대상이었다.

5. 앞으로의 방향

이력을 근거로 몇 가지 방향이 분명해진다.

5-1. 단기 (이미 설계서에 있거나 시작된 것)

- **인덱스·제약조건 수집.** 현재 수집은 테이블·컬럼까지만 훑는다. 인덱스와 제약을 같이 수집해야 구조 진단이 완전해진다.

- **구조 진단의 안정화.** Phase 4에서 재설계가 한 번 더 들어갔지만 아직 "바로 끝났다"고 말하기 어렵다. 수집과의 경계, UI 플로우, 결과 저장 위치를 더 정리해야 한다.
- **승인의 RBAC 실효화.** 역할 기반 권한은 설계·구현됐지만, 기능별로 일관적으로 강제되고 있는지 블랙박스 테스트에서 몇 건 의문이 있었다. 점검이 필요하다.
- **DSTerm.vue 리팩토링.** 2,350줄짜리 단일 컴포넌트. 추가/수정 모달 코드 중복 90%+. TermForm을 분리해서 재사용하는 작업. 일정에는 올라갔지만 아직 착수 전.

5-2. 중기 (반드시 다음 사이클에 다뤄야 하는 것)

- **DQ(데이터 품질) 함수의 부활.** 설계상 존재했지만 비활성화된 상태. 값 수준의 품질 지표(null 비율, 유니크 비율, 형식 위반 등)를 계산하는 축. 이게 들어가야 "표준 준수율"만 있는 상태에서 "데이터 품질"로 시야가 넓어진다.
- **스케줄 수집.** 현재는 수동 트리거. 주기 수집·알림이 없으면 "운영 중 drift"를 잡을 수 없다.
- **학습형 추천.** 자동 표준화의 추천 결과를 사용자가 수정하면 그 수정을 기록해서 다음 추천에 반영한다. 예: "정산"에 대한 추천이 초기 STLM → 사용자가 계속 ADJUST로 수정 → 다음부터 ADJUST가 우선. 설계서에는 Phase 2 확장으로 적어뒀지만 아직 착수 전.
- **대시보드의 drill-down.** 현재 대시보드는 요약 숫자와 추이 차트까지다. 각 숫자를 클릭했을 때 해당하는 컬럼 목록/테이블 목록으로 바로 들어가는 drill-down이 없다. 운영 도구로서 꼭 필요한 축.

5-3. 장기 (방향성은 정했지만 착수 미정)

- **계보 분석(Lineage).** 컬럼 A가 어떤 쿼리/뷰/프로시저를 거쳐 컬럼 B로 흘러가는지 추적. 경쟁사가 강한 영역이고 dataQ가 약한 영역.
- **외부 시스템 연동.** ERwin 임포트는 Phase 2에서 넣었지만 양방향은 아직. Talend/Airflow 같은 ETL 도구와의 연동, CI/CD에 표준 검증 단계 넣기.
- **멀티 테넌트.** 현재는 단일 조직 전제. 여러 조직이 각자의 표준사전을 가지고 dataQ를 공유하는 운영 모델은 아직 없음.
- **AI 기반 자동 추천.** 현재 자동 표준화는 규칙 기반(점수 + DP). 동일한 문제를 LLM으로 풀면 영문명 제안·도메인 추천 품질이 달라질 수 있다. 리소스와 폐쇄망 제약을 고려해야 함.